RELATÓRIO DO TRABALHO PRÁTICO

SISTEMAS OPERATIVOS

Licenciatura em Engenharia de Sistemas Informáticos

Aluno

Ana Rita Ranito (a31511)

Érica Segredo (a30623)

Filipe Fernandes (a31498)

Ruben Costa (a23508)

Professor

Nuno Lopes

maio de 2025

LISTA ABREVIATURAS E/OU SIGLAS

BASH - Bourne Again Shell

EST – Escola Superior de Tecnologia

FAT – File Allocation Table

IPCA – Instituto Politécnico do Cávado e do Ave

LESI - Licenciatura em Engenharia de Sistemas Informáticos

LVM2 - Logical Volume Manager version 2

MV - Máquinas Virtuais

SO – Sistemas Operativos

UC - Unidade Curricular

POSIX - Portable Operating System Interface

WSL - Windows Subsystem for Linux

INDICE DE FIGURAS

Figura 1- Inserir disco novo no VirtualBox 13
Figura 2 - Inserir disco de 10GB no VirtualBox 13
Figura 3 - Ícone do GParted 14
Figura 4 - Comando para instalar o GParted pela bash 14
Figura 5 - Criação de uma nova partição 14
Figura 6 - Instalação do LVM2 14
Figura 7 - Criação do Volume Físico 14
Figura 8 – Criação do Volume Group 15
Figura 9 - Criação do volume lógico onde vai ser utilizado o ext4 15
Figura 10 -Volume lógico onde vai ser utilizado o ext3 15
Figura 11 - Criação do sistema de ficheiros ext4 no primeiro volume lógico 15
Figura 12 - Criação do sistema de ficheiros ext3 no segundo volume lógico 15
Figura 13 - Criação dos diretórios 15
Figura 14 - Mount do primeiro volume 15
Figura 15 - Mount do segundo volume 15
Figura 16 - Comando que torna a primeira montagem persistente 16
Figura 17 - Comando que torna a segunda montagem persistente 16
Figura 18 - Comando que cria o ficheiro com o número dos elementos do grupo 16
Figura 19 - Comando que muda o dono do ficheiro atribuindo-o ao utilizador atual 16
Figura 20 - Tabela de Permissões de ficheiros em Unix 17
Figura 21 - Comando que configura as permissões do ficheiro conforme solicitado 17
Figura 22 - Comando para Instalação do pacote Sleuth Kit através do gestor de pacotes apt 17
Figura 23 - Comando file fs.img 18
Figura 24 - Output do comando fsstat -f fat12 fs.img 18
Figura 25 - Output do comando fsstat -f fat12 fs.img 18
Figura 26 - Comando fls -f fat12 fs.img 19
Figura 27 - Comando istat -f fat12 fs.img 4 e o seu output 19
Figura 28- Comando para leitura do conteúdo do ficheiro ipca.txt utilizando a ferramenta icat 19

INDICE DE TABELAS

Tabela 1 - Funções implementadas em C 8
Tabela 2 - Tabela resumo 9
Tabela 3 - Funcionamento do Interpretador de comandos.....11

ÍNDICE

LISTA ABREVIATURAS E/OU SIGLAS i

INDICE DE FIGURAS ii

INDICE DE TABELAS iii

INTRODUÇÃO 1

1. IMPLEMENTAÇÃO DE UM CONJUNTO DE COMANDOS PARA MANIPULAÇÃO DE FICHEIROS 2

2. IMPLEMENTAÇÃO DE UM INTERPRETADOR DE LINHA DE COMANDOS 10

3. GESTÃO DE SISTEMAS DE FICHEIROS 13

3.1. ANÁLISE AO SISTEMA DE FICHEIROS 17

CONCLUSÃO 20

REFERÊNCIAS BIBLIOGRÁFICAS 21

ANEXOS 22

ANEXO I – IMAGEM DA PASTA COM OS FICHEIRO 23

ANEXO II - IMAGEM DO CONTEÚDO FICHEIRO TEXTO.TXT 24

ANEXO III – IMAGENS DA EXECUÇÃO DAS FUNÇÕES 25

ANEXO IV - IMAGEM DA EXECUÇÃO INTERPRETADOR.C 27

INTRODUÇÃO

O presente relatório pretende dar resposta a um objetivo de avaliação da Unidade Curricular (UC): Sistemas Operativos (SO), inserida no plano de estudos da Licenciatura em Engenharia de Sistemas Informáticos (LESI), da Escola Superior de Tecnologia (EST), do Instituto Politécnico do Cávado e do Ave (IPCA).

Este relatório objetiva discutir os resultados obtidos no trabalho prático que se subdividia em três partes, visando explorar os diferentes gestores do SO, tendo sido utilizado na elaboração do mesmo a distribuição *Linux*: Ubuntu, quer em máquina virtual através do *VirtualBox*, quer através do *WSL (Windows Subsystem for Linux).*

Já no que concerne ao trabalho, de que este relatório trata, pretende aprofundar a compreensão sobre a importância de cada um dos gestores do SO, trabalhar diretamente com as *system calls,* implementar comandos em linguagem C que sejam capazes de manipular ficheiros, explorar as funcionalidades de partições, permissões, e explorar ainda os diferentes tipos de sistemas de ficheiros. Foram ainda aprofundados os conhecimentos acerca do funcionamento de um interpretador de comandos, tendo o presente trabalho como objetivo a criação de um interpretador e deste modo consecutivamente a manipulação de processos. Para o desenvolvimento deste trabalho foram aplicados os conhecimentos lecionados nas aulas de SO, bem como saberes aprendidos nas UC´s lecionadas no primeiro e segundo semestre de LESI.

O relatório encontra-se dividido em seis partes, das quais: Introdução, Implementação de um conjunto de comandos para manipulação de ficheiros, Implementação de um interpretador de linha de comandos, Gestão de Sistema de Ficheiros, Conclusão e Referencias Bibliográficas. Na Introdução, o presente capítulo, é feita a apresentação do contexto, objetivos e estrutura do relatório. O capítulo denominado “Implementação de um conjunto de comandos para manipulação de ficheiros”, consiste no código fonte dos comandos implementados, utilizando as *system calls*. No capítulo da Implementação de um interpretador de linha de comandos é explorada a gestão de processos, assim como evidenciado o código fonte do interpretador desenvolvido. No capítulo “Gestão de Sistema de Ficheiros” são aprofundados conceitos acerca de partições, Volume Groups, Logical Volumes, sendo ainda também aprofundados conceitos acerca de permissões do SO. Já na Conclusão, temos um resumo dos resultados obtidos e dificuldades sentidas na realização deste trabalho prático. Por último, as Referências Bibliográficas, onde são apresentadas as bases teóricas utilizadas no desenvolvimento do trabalho.

1. IMPLEMENTAÇÃO DE UM CONJUNTO DE COMANDOS PARA MANIPULAÇÃO DE FICHEIROS

Os SO modernos, como o *Linux*, fornecem aos utilizadores e programadores interfaces poderosas para a manipulação de ficheiros e diretórios. Estes mecanismos são implementados ao nível do *kernel* e disponibilizados ao programador através de chamadas ao sistema (*system calls*), permitindo o acesso direto a funcionalidades fundamentais do sistema de ficheiros.

Ao longo do tempo, os sistemas de ficheiros evoluíram de estruturas simples baseadas em blocos sequenciais para sistemas hierárquicos complexos, com suporte a permissões, meta-informação, links simbólicos, entre outros. O Unix introduziu, nos anos 70, um modelo de sistema de ficheiros muito influente, baseado em ficheiros, diretórios e i-nodes, modelo esse que ainda serve de base aos sistemas POSIX (Portable Operating System Interface) atuais, como o *ext4* usado no *Linux*. Essa evolução permitiu uma melhor organização e gestão da informação, contribuindo diretamente para a segurança, fiabilidade e eficiência do armazenamento e acesso a dados (Tanenbaum & Bos, 2015).

As *system calls* são a única forma segura de um programa em modo utilizador interagir com o *kernel*. No caso da manipulação de ficheiros, o *Linux* fornece um conjunto de chamadas como:

- open(), read(), write(), close() – para acesso básico a ficheiros.
- stat() – para obter informações sobre um ficheiro.
- unlink() – para apagar ficheiros.
- opendir(), readdir(), closedir() – para aceder ao conteúdo de diretórios.

Estas chamadas podem ser usadas diretamente na linguagem C, sem recorrer a bibliotecas de alto nível, o que permite um maior controlo sobre o funcionamento do sistema (Love, 2010).

O *Linux* é um Sistema Operativo livre e de código aberto, amplamente utilizado tanto em servidores como em ambientes de desenvolvimento, sistemas embebidos e até em supercomputadores. Pela sua conformidade com o padrão POSIX e por ser *open source*, tornou-se o sistema de eleição para o ensino de SO, permitindo estudar e modificar o

comportamento do sistema a um nível muito próximo do hardware (Silberschatz, Galvin & Gagne, 2018).

Para este trabalho, o *Linux* oferece a infraestrutura ideal para testar comandos com chamadas ao sistema reais, em ambiente de consola e com comportamento previsível e documentado.

A linguagem C é frequentemente usada em SO por ser eficiente, de baixo nível e permitir acesso direto à memória e ao hardware. Foi usada para implementar o *Unix* original, e continua a ser a principal linguagem usada para o desenvolvimento do *kernel Linux* e das bibliotecas de sistema (Kernighan & Ritchie, 1988).

Neste trabalho, todos os comandos foram implementados em C, com chamadas diretas ao sistema, sem o uso de funções de biblioteca C como fopen() ou fwrite(), promovendo uma aprendizagem prática do funcionamento real do SO. Podemos ver na Tabela 1 as funções implementadas.

mostra.c

```c
#include <stdio.h>
#include <fcntl.h>      // open
#include <unistd.h>     // read, write, close
#include <stdlib.h>     // exit
#include <errno.h>      // errno
#include <string.h>     // strerror

#define BUF_SIZE 1024

int main(int argc, char *argv[]) {
    int fd;
    ssize_t bytes_read;
    char buffer[BUF_SIZE];

    // Verifica se foi passado o nome do ficheiro
    if (argc != 2) {
        write(2, "Uso: mostra <ficheiro>\n", 24);
        exit(1);
    }

    // Abre o ficheiro em modo só leitura
    fd = open(argv[1], O_RDONLY);
    if (fd == -1) {
        // Mensagem de erro no stderr
        dprintf(2, "Erro ao abrir ficheiro '%s': %s\n", argv[1], strerror(errno));
        exit(2);
    }

    // Lê e escreve até acabar o ficheiro
    while ((bytes_read = read(fd, buffer, BUF_SIZE)) > 0) {
        if (write(1, buffer, bytes_read) != bytes_read) {
            dprintf(2, "Erro ao escrever para stdout\n");
            close(fd);
            exit(3);
        }
    }

    if (bytes_read < 0) {
        dprintf(2, "Erro ao ler ficheiro '%s'\n", argv[1]);
        close(fd);
        exit(4);
    }

    close(fd);
    return 0;
```

Garante que foi passado exatamente 1 argumento (o nome do ficheiro).
Caso contrário, escreve uma mensagem de erro no stderr (descritor 2).

Abre o ficheiro em modo de leitura (O_RDONLY).
Se falhar, imprime o erro no stderr com mensagem explicativa e strerror(errno).

Lê do ficheiro para o buffer até acabar.
Escreve no stdout (descritor 1).
Se falhar a escrita, imprime mensagem no stderr e termina.

- Lê um ficheiro passado como argumento na linha de comandos.
- Imprime o conteúdo desse ficheiro no stdout (ecrã).
- Usa chamadas baixo nível (sistema) e trata erros com cuidado.

copia.c

```c
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdlib.h>
#include <errno.h>
#include <string.h>

#define BUF_SIZE 1024

int main(int argc, char *argv[]) {
    int fd_src, fd_dest;
    ssize_t bytes;
    char buffer[BUF_SIZE];
    char dest_name[256];

    if (argc != 2) {
        write(2, "Uso: copia <ficheiro>\n", 23);
        exit(1);
    }

    snprintf(dest_name, sizeof(dest_name), "%s.copia", argv[1]);

    fd_src = open(argv[1], O_RDONLY);
    if (fd_src == -1) {
        dprintf(2, "Erro ao abrir ficheiro origem: %s\n", strerror(errno));
        exit(2);
    }

    fd_dest = open(dest_name, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd_dest == -1) {
        dprintf(2, "Erro ao criar ficheiro destino: %s\n", strerror(errno));
        close(fd_src);
        exit(3);
    }

    while ((bytes = read(fd_src, buffer, BUF_SIZE)) > 0) {
        if (write(fd_dest, buffer, bytes) != bytes) {
            dprintf(2, "Erro ao escrever no destino\n");
            close(fd_src);
            close(fd_dest);
            exit(4);
        }
    }

    close(fd_src);
    close(fd_dest);
    return 0;
}
```

Validação de argumentos
Verifica se o utilizador passou o nome do ficheiro original.

Criação do nome do novo ficheiro

Abertura do ficheiro de origem.
Se falhar, mostra erro com strerror(errno) e termina com exit(2).

Criar ficheiro de destino
O_WRONLY: escrita
O_CREAT: cria ficheiro se não existir
O_TRUNC: esvazia ficheiro se já existir
0644: permissões (rw-r--r--)

Cópia do conteúdo
Lê do ficheiro original e escreve no novo ficheiro. Se falhar, mostra erro e termina com exit(4).

- Recebe como argumento o nome de um ficheiro existente.
- Copia o conteúdo desse ficheiro para outro com o mesmo nome + .copia no fim.
- Exemplo: original.txt → original.txt.copia

acrescenta.c

```c
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdlib.h>
#include <errno.h>
#include <string.h>

#define BUF_SIZE 1024

int main(int argc, char *argv[]) {
    int fd_src, fd_dest;
    ssize_t bytes;
    char buffer[BUF_SIZE];

    if (argc != 3) {
        write(2, "Uso: acrescenta <origem> <destino>\n", 36);
        exit(1);
    }

    fd_src = open(argv[1], O_RDONLY);
    if (fd_src == -1) {
        dprintf(2, "Erro ao abrir origem: %s\n", strerror(errno));
        exit(2);
    }

    fd_dest = open(argv[2], O_WRONLY | O_APPEND);
    if (fd_dest == -1) {
        dprintf(2, "Erro ao abrir destino: %s\n", strerror(errno));
        close(fd_src);
        exit(3);
    }

    while ((bytes = read(fd_src, buffer, BUF_SIZE)) > 0) {
        if (write(fd_dest, buffer, bytes) != bytes) {
            dprintf(2, "Erro ao escrever no destino\n");
            close(fd_src);
            close(fd_dest);
            exit(4);
        }
    }

    close(fd_src);
    close(fd_dest);
    return 0;
}
```

Validação de argumentos
Garante que o utilizador forneceu 2 ficheiros:
- argv[1]: ficheiro de origem
- argv[2]: ficheiro de destino

Abre o ficheiro de origem só para leitura. Se falhar, mostra erro e sai.

Abrir ficheiro de destino
-O_WRONLY: apenas para escrita
-O_APPEND: escreve no fim do ficheiro, sem apagar o conteúdo anterior

Lê dados da origem em blocos de 1024 bytes.
Escreve no final do ficheiro de destino.
Se a escrita falhar, fecha os descritores e sai com erro.

- Lê o conteúdo de um ficheiro de origem (argumento 1).
- Acrescenta esse conteúdo ao final de um ficheiro de destino (argumento 2).
- Usa chamadas baixo nível: open, read, write, close.

conta.c

```c
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdlib.h>

#define BUF_SIZE 1024

int main(int argc, char *argv[]) {
    int fd;
    ssize_t bytes;
    char buffer[BUF_SIZE];
    int linhas = 0;

    if (argc != 2) {
        write(2, "Uso: conta <ficheiro>\n", 23);
        exit(1);
    }

    fd = open(argv[1], O_RDONLY);
    if (fd == -1) {
        perror("Erro ao abrir ficheiro");
        exit(2);
    }

    while ((bytes = read(fd, buffer, BUF_SIZE)) > 0) {
        for (ssize_t i = 0; i < bytes; i++) {
            if (buffer[i] == '\n') linhas++;
        }
    }

    dprintf(1, "Número de linhas: %d\n", linhas);

    close(fd);
    return 0;
}
```

- Ler um ficheiro passado como argumento e contar quantas linhas tem (ou seja, quantas vezes o caracter '\n' aparece).

Declaração de variáveis
-fd: descritor do ficheiro
-bytes: número de bytes lidos em cada iteração
-buffer: armazenamento temporário de conteúdo lido
-linhas: contador de quebras de linha \n

Verificação de argumentos
Garante que o utilizador passou um nome de ficheiro como argumento.

Lê o ficheiro em blocos de 1024 bytes. Para cada byte lido, verifica se é um '\n'. Se for, incrementa o contador de linhas.

Imprime o número total de linhas no stdout (descritor 1).

apaga.c

```c
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <errno.h>
#include <string.h>

int main(int argc, char *argv[]) {
    if (argc != 2) {
        write(2, "Uso: apaga <ficheiro>\n", 23);
        exit(1);
    }

    if (unlink(argv[1]) == -1) {
        dprintf(2, "Erro ao apagar ficheiro: %s\n", strerror(errno));
        exit(2);
    }

    write(1, "Ficheiro apagado com sucesso\n", 30);
    return 0;
}
```

- Recebe o nome de um ficheiro como argumento da linha de comandos.
- Tenta apagá-lo com unlink().
- Imprime mensagem de sucesso ou erro.

Verificação de argumentos
Garante que o utilizador passou exatamente um argumento: o nome do ficheiro a apagar. Se não passar, escreve erro no stderr (descritor 2) e termina.

unlink() é a chamada ao sistema que remove o ficheiro do sistema de ficheiros. Se falhar imprime uma mensagem no stderr.

informa.c

```c
#include <unistd.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <time.h>
#include <stdio.h>
#include <string.h>

void escreve_info(struct stat *info) {
    char buffer[256];
    char data_str[100];
    struct tm *tm;

    // Tipo de ficheiro
    if (S_ISREG(info->st_mode)) {
        write(STDOUT_FILENO, "Tipo: Ficheiro normal\n", 23);
    } else if (S_ISDIR(info->st_mode)) {
        write(STDOUT_FILENO, "Tipo: Diretoria\n", 17);
    } else if (S_ISLNK(info->st_mode)) {
        write(STDOUT_FILENO, "Tipo: Link simbólico\n", 22);
    } else {
        write(STDOUT_FILENO, "Tipo: Outro tipo de ficheiro\n", 30);
    }

    // i-node
    snprintf(buffer, sizeof(buffer), "i-node: %ld\n", (long)info->st_ino);
    write(STDOUT_FILENO, buffer, strlen(buffer));

    // Utilizador dono
    snprintf(buffer, sizeof(buffer), "Utilizador dono: %d\n", info->st_uid);
    write(STDOUT_FILENO, buffer, strlen(buffer));

    // Data de criação
    tm = localtime(&info->st_ctime);
    strftime(data_str, sizeof(data_str), "%Y-%m-%d %H:%M:%S", tm);
    snprintf(buffer, sizeof(buffer), "Data de criação: %s\n", data_str);
    write(STDOUT_FILENO, buffer, strlen(buffer));

    // Data de última leitura
    tm = localtime(&info->st_atime);
    strftime(data_str, sizeof(data_str), "%Y-%m-%d %H:%M:%S", tm);
    snprintf(buffer, sizeof(buffer), "Data de última leitura: %s\n", data_str);
    write(STDOUT_FILENO, buffer, strlen(buffer));

    // Data de última modificação
    tm = localtime(&info->st_mtime);
    strftime(data_str, sizeof(data_str), "%Y-%m-%d %H:%M:%S", tm);
    snprintf(buffer, sizeof(buffer), "Data de última modificação: %s\n", data_str);
    write(STDOUT_FILENO, buffer, strlen(buffer));
}

int informa_ficheiro(const char *nome_ficheiro) {
    struct stat info;

    if (stat(nome_ficheiro, &info) < 0) {
        write(STDERR_FILENO, "Erro: ficheiro não encontrado\n", 31);
        return 1;
    }

    escreve_info(&info);
    return 0;
}

int main(int argc, char *argv[]) {
    if (argc != 2) {
        write(STDERR_FILENO, "Uso: informa ficheiro <nome>\n", 30);
        return 1;
    }
    return informa_ficheiro(argv[1]);
}
```

Recebe um ponteiro para uma estrutura stat e imprime:

- Tipo de ficheiro;
  Normal (S_ISREG)
  Diretoria (S_ISDIR)
  Link simbólico (S_ISLNK)
  Outro tipo
- i-node;
- id do dono;
- Data de "criação" (na verdade st_ctime)*;
- Última leitura (st_atime);
- Última modificação (st_mtime).

Preenche a estrutura stat com os dados do ficheiro. Se stat() falhar, mostra erro. Caso contrário, chama escreve_info() com os dados recolhidos.

Verifica se foi passado exatamente 1 argumento (nome do ficheiro).
Se sim, chama informa_ficheiro (argv[1]).

Imprime no terminal informações detalhadas sobre um ficheiro fornecido como argumento:

- Tipo de ficheiro (normal, diretoria, link simbólico, etc.)
- i-node
- UID do utilizador dono
- Datas de criação, última leitura e última modificação

*Em Linux, st_ctime não é data de criação! É a data da última alteração dos metadados (como permissões). O Linux, por norma, não guarda a data de criação.

lista.c

```c
#include <unistd.h>
#include <stdio.h>
#include <dirent.h>
#include <sys/stat.h>
#include <string.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
    DIR *d;
    struct dirent *dir;
    struct stat st;
    char caminho[512];
    char *pasta = ".";

    if (argc == 2) {
        pasta = argv[1];
    } else if (argc > 2) {
        write(2, "Uso: lista [diretoria]\n", 24);
        exit(1);
    }

    d = opendir(pasta);
    if (!d) {
        perror("Erro ao abrir diretoria");
        exit(2);
    }

    while ((dir = readdir(d)) != NULL) {
        snprintf(caminho, sizeof(caminho), "%s/%s", pasta, dir->d_name);
        if (stat(caminho, &st) == 0) {
            if (S_ISDIR(st.st_mode))
                printf("%s\t[diretoria]\n", dir->d_name);
            else if (S_ISREG(st.st_mode))
                printf("%s\t[ficheiro]\n", dir->d_name);
        }
    }

    closedir(d);
    return 0;
}
```

Declaração de variáveis
-ponteiro para diretoria aberta
-entrada da diretoria
-estrutura com info do ficheiro
-caminho completo para cada entrada
-diretoria padrão (atual)

Se for passado um argumento, usa-o como diretoria.
Caso contrário, assume "." (diretoria atual).

Para cada entrada
1-Constrói o caminho completo
2- Usa stat() para obter informação sobre essa entrada.
3-Verifica se é diretório ou ficheiro regular e imprime.

- Recebe como argumento uma diretoria (ou assume "." se não for passado nenhum argumento).
- Percorre o conteúdo da diretoria.
- Imprime cada entrada, identificando se é:
  - [diretoria]
  - [ficheiro]

Tabela 1 - Funções implementadas em C

Objetivando perceber melhor as funcionalidades das funções apresentadas na Tabela 1, foi elaborada uma Tabela resumo (Tabela 2).

Ficheiro	Principal função	Operações Realizadas	Chamadas ao Sistema Usadas
mostra.c	Lê e mostra no ecrã o conteúdo de um ficheiro.	• valida argumentos; • abre ficheiro; • lê e escreve no stdout; • trata erros.	• open; • read; • write; • close.
copia.c	Copia um ficheiro para outro com o sufixo .copia.	• valida argumentos; • cria nome novo; • abre/cria ficheiros; • lê e escreve; • trata erros.	• open; • read; • write; • close.
acrescenta.c	Acrescenta o conteúdo de um ficheiro no final de outro.	• valida 2 argumentos; • abre ambos os ficheiros; • lê e escreve em modo append.	• open; • read; • write; • close.
conta.c	Conta o número de linhas de um ficheiro.	• valida argumentos; • lê ficheiro em blocos; • conta \n; • imprime resultado.	• open; • read; • write; • close.
apaga.c	Apaga um ficheiro passado como argumento.	• verifica argumentos; • chama unlink() para remover o ficheiro; • imprime sucesso/erro.	• unlink.
informa.c	Mostra informações detalhadas de um ficheiro.	• usa stat() para obter: tipo, inode, UID, datas (st_ctime, st_atime, st_mtime); • imprime resultado formatado.	• stat.
lista.c	Lista entradas de uma diretoria (ou a atual), identificando ficheiros e pastas.	• usa opendir; • percorre entradas; • usa stat para verificar tipo (ficheiro ou diretoria); • imprime por tipo.	• opendir; • readdir; • stat.

Tabela 2 - Tabela resumo

Deste modo, todos os ficheiros executáveis resultantes foram colocados no mesmo diretório, assim como um ficheiro de texto de teste destas funções, conforme é possível constatar no ANEXO I e ANEXO II. Todas as funções foram testadas, os resultados das suas execuções encontram-se representadas no ANEXO III.

2. IMPLEMENTAÇÃO DE UM INTERPRETADOR DE LINHA DE COMANDOS

O Bash (Bourne Again Shell) é o interpretador de comandos padrão na maioria das distribuições *Linux*. Ele permite a execução interativa de comandos e scripts, fornecendo um ambiente poderoso para automação de tarefas (Shotts, 2019). No contexto deste trabalho, foi desenvolvido um mini-interpretador que se comporta de forma semelhante ao Bash, executando comandos por meio de fork() e exec(), e capturando o seu código de saída com wait(). Este exercício ajuda a compreender o funcionamento interno da *bash* e a forma como o *Linux* cria e gere processos.

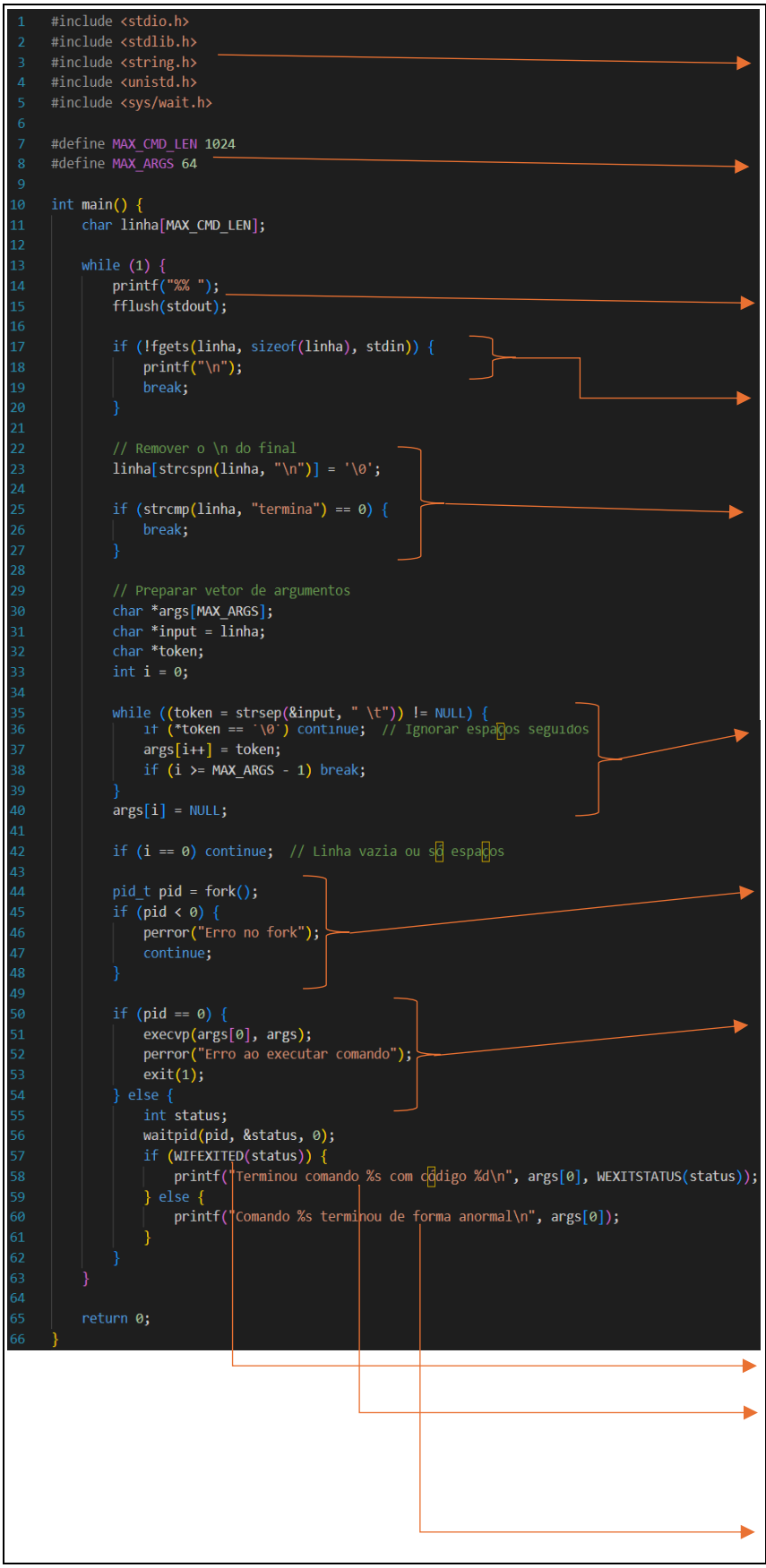
Deste modo foram utilizados vários comandos de modo a gerir estes processos, nomeadamente a função fork() que é utilizada para criar um processo, duplicando o processo atual (processo pai). O novo processo criado é chamado de processo filho. Após a chamada de fork(), ambos os processos continuam a executar a partir do ponto imediatamente seguinte à chamada, com a diferença de que fork() retorna 0 no processo filho e o PID do filho no processo pai (Love, 2010).

O processo filho normalmente utiliza a função exec() (ou uma das suas variantes, como execvp()) para substituir a sua imagem de processo atual por um novo programa. Esta chamada carrega o programa indicado e começa a sua execução, substituindo completamente o código anterior. Assim, o processo filho torna-se o novo programa (Kerrisk, 2010). A função execvp() é uma função da família exec, definida em <unistd.h>, usada para substituir o processo atual por outro programa, sendo a assinatura desta função: int execvp(const char *file, char *const argv[]). Esta recebe como parâmetros o nome do programa a ser executado, e um array de strings com os argumentos a passar ao programa. Este execvp procura o programa no *PATH*. O *PATH* é uma variável de ambiente no Unix/Linux que contém uma lista de diretórios onde o sistema vai procurar executáveis. Quando invocamos o execvp este procura dentro dos diretórios listados no *PATH*, e deste modo o sistema não exige o caminho completo.

Entretanto, o processo pai utiliza a função wait() (ou waitpid()) para aguardar que o processo filho termine a sua execução. Esta função permite ao processo pai obter o código de saída do filho, indicando se terminou com sucesso ou se ocorreu algum erro.

Este exercício ajuda a compreender o funcionamento interno da bash e a forma como o *Linux* cria e gere processos, destacando a importância da separação entre pai e filho, bem como o controlo explícito da execução de comandos através destas chamadas de sistema.

Consecutivamente foi então desenvolvido o seguinte interpretador de comandos na linguagem C, como é possível evidenciar na Tabela 3:



```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_CMD_LEN 1024
#define MAX_ARGS 64

int main() {
    char linha[MAX_CMD_LEN];

    while (1) {
        printf("%% ");
        fflush(stdout);

        if (!fgets(linha, sizeof(linha), stdin)) {
            printf("\n");
            break;
        }

        // Remover o \n do final
        linha[strcspn(linha, "\n")] = '\0';

        if (strcmp(linha, "termina") == 0) {
            break;
        }

        // Preparar vetor de argumentos
        char *args[MAX_ARGS];
        char *input = linha;
        char *token;
        int i = 0;

        while ((token = strsep(&input, " \t")) != NULL) {
            if (*token == '\0') continue;  // Ignorar espaços seguidos
            args[i++] = token;
            if (i >= MAX_ARGS - 1) break;
        }
        args[i] = NULL;

        if (i == 0) continue;  // Linha vazia ou só espaços

        pid_t pid = fork();
        if (pid < 0) {
            perror("Erro no fork");
            continue;
        }

        if (pid == 0) {
            execvp(args[0], args);
            perror("Erro ao executar comando");
            exit(1);
        } else {
            int status;
            waitpid(pid, &status, 0);
            if (WIFEXITED(status)) {
                printf("Terminou comando %s com código %d\n", args[0], WEXITSTATUS(status));
            } else {
                printf("Comando %s terminou de forma anormal\n", args[0]);
            }
        }
    }

    return 0;
}
```

Inclusão de bibliotecas necessárias para chamadas ao sistema, entrada/saída e manipulação de strings.

Definições para o tamanho máximo da linha de comando e número de argumentos.

Mostra o prompt personalizado % e espera o input do utilizador.

Usa fgets() para ler a linha de comando da consola.
Se for EOF (Ctrl+D), termina.

Remove o \n no fim da linha.
Se o comando for termina, o interpretador termina a execução.

Usa strsep() para separar a linha em tokens (comando + argumentos).
Ignora múltiplos espaços/tabulações consecutivas.
O vetor args[] é construído para passar ao execvp().

Cria um processo filho com fork().
Se falhar, mostra erro e volta ao início.

Processo filho executa o comando com execvp().
Se falhar (comando não encontrado, erro de execução...), imprime mensagem de erro e termina com exit(1).

O processo pai espera que o filho termine com waitpid().

Depois do filho terminar:

Usa WIFEXITED() para verificar se terminou normalmente;
Usa WEXITSTATUS() para obter o código de saída;

Imprime mensagem ao utilizador com o resultado da execução.

Tabela 3 - Funcionamento do Interpretador de comandos

De um modo sintetizado, o interpretador de comandos:

- Mostra o prompt “%” para o utilizador escrever um comando;
- Lê a linha de comando;
- Se o comando for "termina", o programa encerra;
- Divide a linha em argumentos (usando espaços e tabs como separadores);
- Cria um processo filho com fork();
- O processo filho executa o comando com execvp();
- O processo pai espera o filho terminar e mostra o código de saída ou se terminou anormalmente;
- Repete este ciclo até o utilizador pedir para terminar.

A execução do interpretador de comandos encontra-se representado no ANEXO IV.

3. GESTÃO DE SISTEMAS DE FICHEIROS

De modo a responder ao enunciado foram utilizadas Máquinas Virtuais (MV), através do *software VirtualBox*, tendo sido criada uma máquina virtual com a distribuição *Linux* Ubuntu.

Cada MV funciona como um sistema independente, permitindo, entre muitos outros: trabalhar com diversos sistemas de ficheiros; criar, formatar e montar discos virtuais de forma segura; testar comandos de administração sem o risco de danificar a máquina real (Silberschatz, Galvin & Gagne, 2018). Permite ainda uma fácil restauração e recuperação, sendo possível manipular e realizar testes com comandos críticos como rm, mkfs, mount, entre outros e explorar diversas configurações de sistema de ficheiros ou permissões (Oracle VM VirtualBox User Manual – Section 1.9 "Snapshots").

Posteriormente foi adicionado um disco com 10GB conforme solicitado, de modo a criar as partições neste disco foi utilizada a aplicação GParted, conforme é possível visualizar nas Figura 1 e Figura 2.

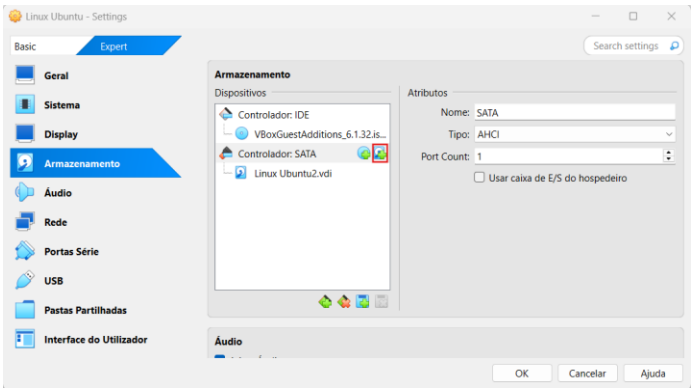


Figura 1- Inserir disco novo no VirtualBox

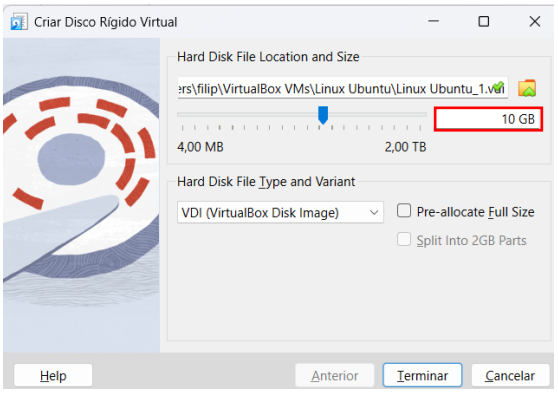


Figura 2 - Inserir disco de 10GB no VirtualBox

Foi instalado o GParted (Figura 3 e Figura 4) (GNOME Partition Editor) pela linha de comandos, este é uma ferramenta gráfica livre e *open-source* utilizada para gerir partições de disco em sistemas *Linux* e outros SO compatíveis. Permite realizar operações como criação, redimensionamento, formatação e eliminação de partições, de forma intuitiva e segura, sem necessidade de recorrer à linha de comandos (GParted Project, 2024)

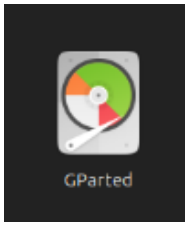


Figura 3 - Ícone do GParted

```
rita@rita-VirtualBox:~$ sudo apt install gparted
```

Figura 4 - Comando para instalar o GParted pela bash

Posteriormente no Gparted foi selecionado o disco que tínhamos acabado de criar, foi também criada uma tabela de partições. Em seguida foi criada uma partição e alocado todo o tamanho do disco a esta partição, como é possível ver na Figura 5.



Figura 5 - Criação de uma nova partição

Os passos seguintes foram executados na *bash*, de modo a corresponder ao enunciado proposto, foi instalado o LVM2 (Figura 6). O LVM2 (Logical Volume Manager version 2) é uma camada de abstração entre os dispositivos de armazenamento físico e os volumes usados pelo SO, permitindo uma gestão mais flexível do espaço em disco. Em vez de depender de partições fixas, o LVM permite criar, redimensionar e mover volumes lógicos conforme necessário. Esta abordagem facilita a adaptação do sistema a novas exigências de armazenamento sem interrupções significativas (Ward, 2014; TLDP, n.d.).

```
rita@rita-VirtualBox:~$ sudo apt install lvm2
```

Figura 6 - Instalação do LVM2

Seguidamente foi inicializada a partição como um Volume Físico, seguido da criação de um Volume Group e criados dois volumes lógicos de 5GB cada. Em cada um destes volumes lógicos foram definidos sistemas de ficheiros diferentes, no primeiro foi definido o sistema ext4 e no segundo o ext3 como é possível observar nas Figura 7, Figura 8, Figura 9, Figura 10, Figura 11 e Figura 12.

```
rita@rita-VirtualBox:~$ sudo pvcreate /dev/sdb1
```

Figura 7 - Criação do Volume Físico

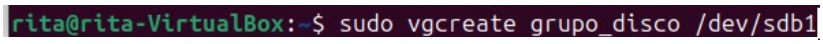


Figura 8 – Criação do Volume Group

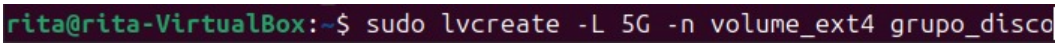


Figura 9 - Criação do volume lógico onde vai ser utilizado o ext4

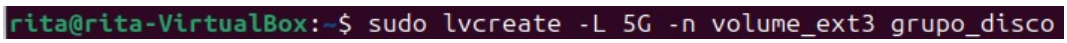


Figura 10 -Volume lógico onde vai ser utilizado o ext3

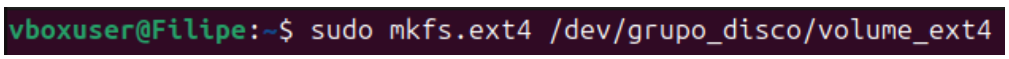


Figura 11 - Criação do sistema de ficheiros ext4 no primeiro volume lógico

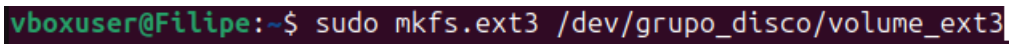


Figura 12 - Criação do sistema de ficheiros ext3 no segundo volume lógico

O ext4 (Fourth Extended Filesystem) é uma evolução do sistema de ficheiros ext3, amplamente utilizado em sistemas *Linux* devido à sua robustez, escalabilidade e desempenho. Introduzido oficialmente no kernel *Linux* 2.6.28 em 2008, o ext4 suporta volumes até 1 exabyte e ficheiros individuais até 16 terabytes. O ext4 é compatível com ext3, permitindo uma transição suave entre sistemas. Além disso, oferece suporte a *journaling*, o que garante maior resiliência contra falhas e corrupção de dados em caso de falhas de energia ou desligamentos inesperados (Mathur et al., 2007; Bovet & Cesati, 2005; Love, 2010).

De seguida foram criados diretórios para a posterior montagem dos volumes e realizada então a montagem com o comando *mount* através de permissões de administrador (Figura 13, Figura 14 e Figura 15).

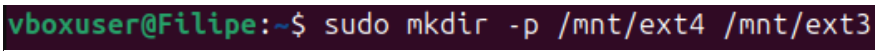


Figura 13 - Criação dos diretórios



Figura 14 - Mount do primeiro volume



Figura 15 - Mount do segundo volume

Posteriormente foram executados os seguintes comandos (Figura 16 e Figura 17) com o intuito de tornar as montagens persistentes:

```
vboxuser@Filipe:~$ echo '/dev/grupo_disco/volume_ext3 /mnt/ext3 ext3 defaults 0 2' | sudo tee -a /etc/fstab
```

Figura 17 - Comando que torna a segunda montagem persistente

```
vboxuser@Filipe:~$ echo '/dev/grupo_disco/volume_ext4 /mnt/ext4 ext4 defaults 0 2' | sudo tee -a /etc/fstab
```

Figura 16 - Comando que torna a primeira montagem persistente

Estes comandos servem para acrescentar entradas ao ficheiro /etc/fstab, que é lido automaticamente no arranque do sistema para montar sistemas de ficheiros.

O comando "echo" gera a string que representa a linha que irá ser adicionada ao ficheiro /etc/fstab. A opção tee -a escreve no ficheiro com acrescento (-a = append), sudo é necessário porque o ficheiro /etc/fstab só pode ser alterado por administradores.

Consecutivamente foi criado o ficheiro com o número dos elementos do grupo no interior da diretoria /mnt/ext4 (Figura 18).

```
vboxuser@Filipe:~$ sudo touch /mnt/ext4/31498-31511.txt
```

Figura 18 - Comando que cria o ficheiro com o número dos elementos do grupo

De modo a corresponder ao seguinte ponto do trabalho prático era solicitado que manuseássemos permissões de ficheiros. As permissões garantem que os utilizadores não consigam aceder nem modificar ficheiros alheios, intencionalmente ou por erro. Isto é especialmente relevante em ambientes multiutilizador, como servidores ou sistemas UNIX/*Linux* (Tanenbaum, 2015). Estas permissões impedem que utilizadores comuns apaguem ficheiros de sistema, modifiquem configurações críticas e acedam a áreas de memória ou dispositivos sem autorização (Silberschatz, Galvin, & Gagne, 2018).

Primeiramente, foi alterado o dono do ficheiro para o utilizador atual, utilizando a variável de ambiente $USER e do comando *chown (change owner)*(Figura 19).

```
vboxuser@Filipe:~$ sudo chown $USER /mnt/ext4/31498-31511.txt
```

Figura 19 - Comando que muda o dono do ficheiro atribuindo-o ao utilizador atual

Em seguida foram alteradas as permissões de modo a fornecer permissões de escrita e leitura para o dono, retirar todas as permissões ao grupo, e a garantir a todos os outros permissões de leitura, tendo sido utilizado o comando evidenciado na Figura 21, obtido através da tabela presente na Figura 20.

Permissão	Binário	Decimal
---	000	0
--x	001	1
-w-	010	2
-wx	011	3
r--	100	4
r-x	101	5
rw-	110	6
rwx	111	7

Figura 20 - Tabela de Permissões de ficheiros em Unix

```
vboxuser@Filipe:~$ chmod 604 /mnt/ext4/31498-31511.txt
```

Figura 21 - Comando que configura as permissões do ficheiro conforme solicitado

3.1. ANÁLISE AO SISTEMA DE FICHEIROS

A análise aos sistemas de ficheiros permite compreender a forma como os dados são organizados, armazenados e acedidos num dispositivo de armazenamento. No contexto desta atividade, procedeu-se à análise de uma imagem de disco (fs.img) formatada com o sistema FAT12 (File Allocation Table, 12 bits). Este sistema de ficheiros foi amplamente utilizado em disquetes e sistemas embebidos devido à sua simplicidade e baixa exigência de recursos. O objetivo principal desta análise foi identificar o bloco de início dos ficheiros regulares presentes no sistema de ficheiros e visualizar o conteúdo do ficheiro com o nome ipca.txt.

Para realizar a análise, foi utilizada a suíte de ferramentas *Sleuth Kit*, uma coleção de utilitários de análise forense de sistemas de ficheiros. Esta suíte inclui ferramentas como *fls*, *fsstat* e *icat*, que permitem explorar imagens de disco de forma detalhada, sem necessidade de montagem.

A instalação das ferramentas foi realizada através do seguinte comando no terminal de uma distribuição *Linux* Ubuntu (Figura 22).

```
erica@erica-VirtualBox:~$ sudo apt install sleuthkit
```

Figura 22 - Comando para Instalação do pacote Sleuth Kit através do gestor de pacotes apt

O primeiro passo consistiu na verificação do tipo de sistema de ficheiros presente na imagem. Para isso, utilizou-se o comando *file* (Figura 23). Este comando serve para identificar o tipo de conteúdo de um ficheiro, analisando os seus dados internos, não apenas a extensão. O ficheiro "*fs.img*" representa uma imagem de um sistema de ficheiros completo. Este ficheiro contém a estrutura hierárquica de diretórios, ficheiros e os

respetivos meta-dados, revelando tratar-se de um sistema de ficheiros FAT12 após termos utilizado o comando previamente mencionado.

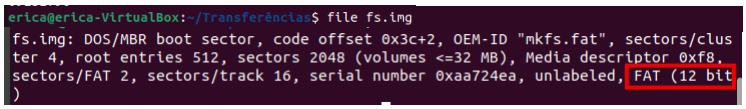


Figura 23 - Comando file fs.img

Em seguida, para obter detalhes estruturais sobre o sistema de ficheiros, foi utilizado o comando *fsstat* (Figura 25):

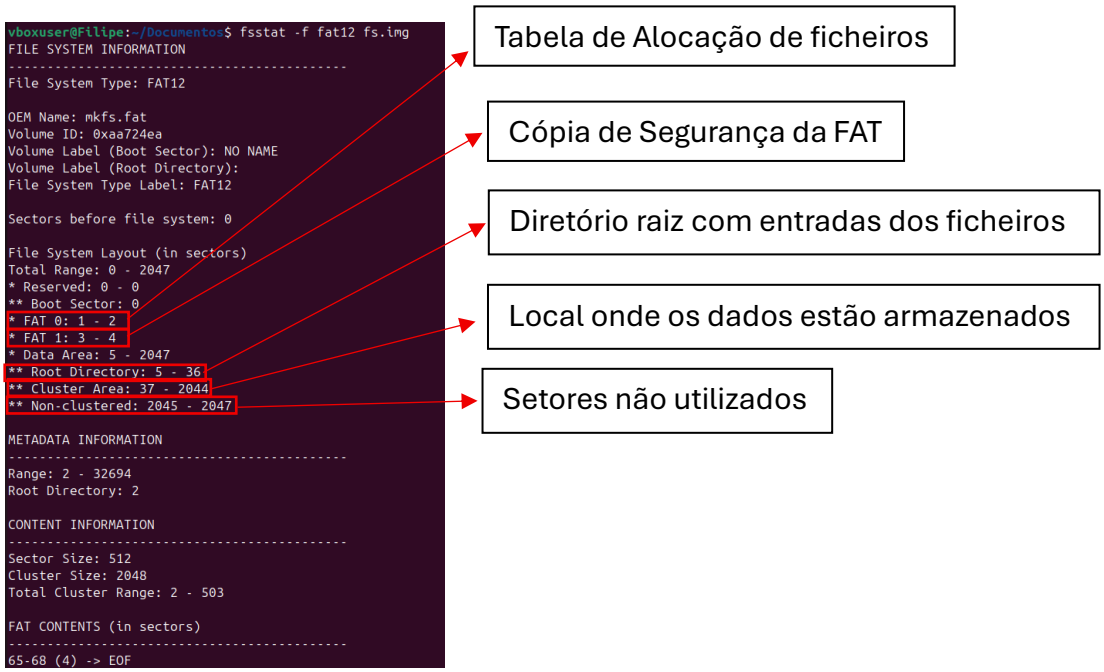


Figura 25 - Output do comando fsstat -f fat12 fs.img

O comando "fsstat -f fat12 fs.img" serve como uma ferramenta de analise forense, examinando a imagem do sistema de ficheiros "fs.img" para fornecer informação detalhada sobre a sua estrutura interna. Esta análise inclui detalhes como o tamanho dos blocos, localização dos dados, setores não utilizados entre outros, assim como outros meta-dados relevantes. Podemos constatar que cada setor possui 512 bytes, e que cada cluster ocupa 4 setores (2048 bytes).

Posteriormente, de modo a listar os ficheiros e diretórios presentes nesta imagem de sistema de ficheiros foi utilizado o comando fls(*File List)*, conforme podemos constatar na Figura 26.

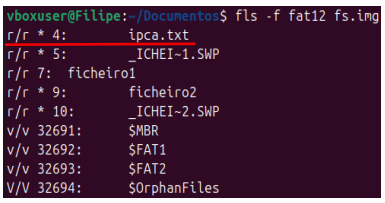


Figura 26 - Comando fls -f fat12 fs.img

Foi possível observar que 4 é o número da entrada FAT (semelhante aos i-nodes), o * simboliza que o ficheiro está apagado ou não alocado.

Posteriormente com o intuito de obter detalhes acerca deste ficheiro "ipca.txt" foi utilizado o seguinte comando (Figura 27):

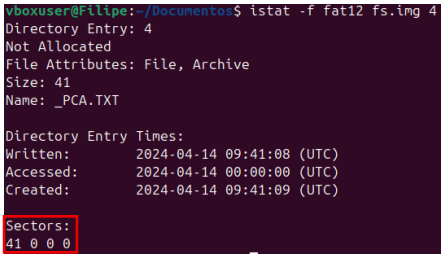


Figura 27 - Comando istat -f fat12 fs.img 4 e o seu output

Podemos verificar que o ficheiro foi apagado. Ainda existe no diretório raiz, mas o sistema já não o considera alocado. Contudo, o conteúdo pode ser recuperado porque os setores não foram sobrescritos. Através da informação acerca dos setores do ficheiro, verificamos que o conteúdo deste começa no setor 41 da imagem fs.img

E por fim, para visualizar o conteúdo do ficheiro ipca.txt, foi utilizado o comando *icat* (Figura 28):

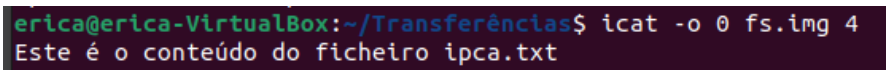


Figura 28- Comando para leitura do conteúdo do ficheiro ipca.txt utilizando a ferramenta icat

Este comando é utilizado para extrair o conteúdo de um ficheiro diretamente de uma imagem de disco, com base no número da entrada FAT (ou inode, em sistemas ext).

Foi possível observar que o conteúdo do ficheiro é:" Este é o conteúdo do ficheiro ipca.txt".

CONCLUSÃO

A realização deste trabalho prático permitiu-nos consolidar conhecimentos essenciais no âmbito da UC de SO, contribuindo significativamente para a compreensão aprofundada do funcionamento dos diversos gestores que compõem um SO. Através da implementação de comandos em linguagem C recorrendo a *system calls*, foi possível perceber a importância do contacto direto com o núcleo do sistema, bem como o papel crucial que as chamadas ao sistema desempenham na interação entre o utilizador e o hardware.

A construção de um interpretador de comandos simples possibilitou o desenvolvimento de competências relacionadas com a gestão de processos, reforçando a compreensão do ciclo de vida de um processo, das suas relações hierárquicas e da comunicação entre processos. Do mesmo modo, a exploração dos conceitos associados à gestão do sistema de ficheiros, como partições, permissões e tipos de sistemas de ficheiros, revelou-se fundamental para compreender a organização e segurança da informação num sistema *Linux*.

Em suma, este trabalho foi uma oportunidade valiosa para aplicar, de forma prática, os conteúdos abordados nas aulas, integrando também conhecimentos transversais adquiridos ao longo dos dois primeiros semestres do curso. Apesar de alguns desafios encontrados ao longo do desenvolvimento, estes serviram como momentos de aprendizagem e reflexão, contribuindo para uma formação mais sólida na área dos SO.

REFERÊNCIAS BIBLIOGRÁFICAS

Aulas SO do Professor Nuno Lopes.

Bovet, D. P., & Cesati, M. (2005). *Understanding the Linux kernel* (3rd ed.). O'Reilly Media.

GParted Project. (2024). *GParted manual*. https://gparted.org/display-doc.php?name=help-manual

Kernighan, B. W., & Ritchie, D. M. (1988). *The C programming language*. Prentice Hall.

Kerrisk, M. (2010). *The Linux programming interface: A Linux and UNIX system programming handbook*. No Starch Press.

Love, R. (2010a). *Linux kernel development* (3rd ed.). Addison-Wesley.

Love, R. (2010b). *Linux system programming*. O'Reilly Media.

Mathur, A., Cao, M., Bhattacharya, S., Dilger, A., Tomas, D., & Vivier, L. (2007). The new ext4 filesystem: Current status and future plans. *Proceedings of the Linux Symposium*, *1*, 21–33. https://www.kernel.org/doc/ols/2007/ols2007v1-pages-21-33.pdf

Oracle Corporation. (n.d.). *Oracle VM VirtualBox user manual – Section 1.9 "Snapshots"*. https://www.virtualbox.org/manual/

Shotts, W. E. (2019). *The Linux command line: A complete introduction*. No Starch Press.

Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating system concepts* (10th ed.). Wiley.

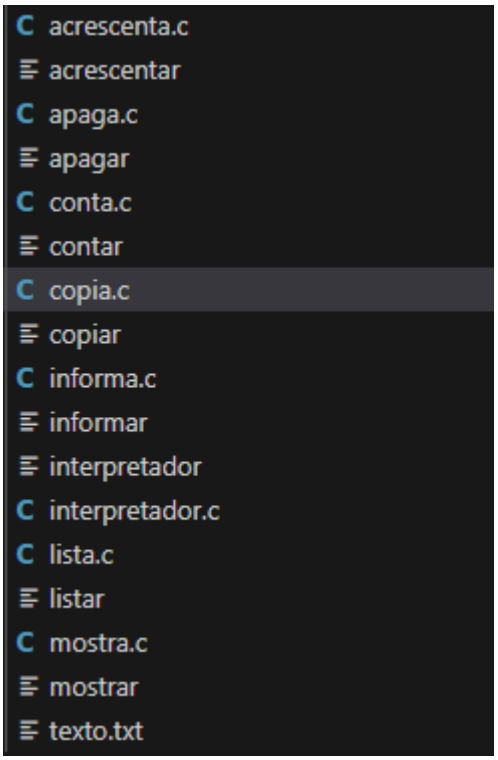
Tanenbaum, A. S., & Bos, H. (2015). *Modern operating systems* (4th ed.). Pearson.

The Linux Documentation Project. (n.d.). *LVM HOWTO*. https://tldp.org/HOWTO/LVM-HOWTO/

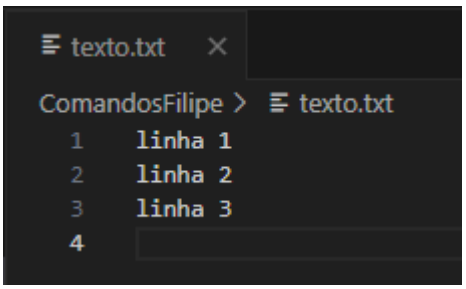
Ward, B. (2014). *Using LVM on Linux*. Red Hat. https://access.redhat.com/documentation/en-us/red_hat_enterprise_linux/7/html/logical_volume_manager_administration/

ANEXOS

ANEXO I – IMAGEM DA PASTA COM OS FICHEIRO



ANEXO II - IMAGEM DO CONTEÚDO FICHEIRO TEXTO.TXT



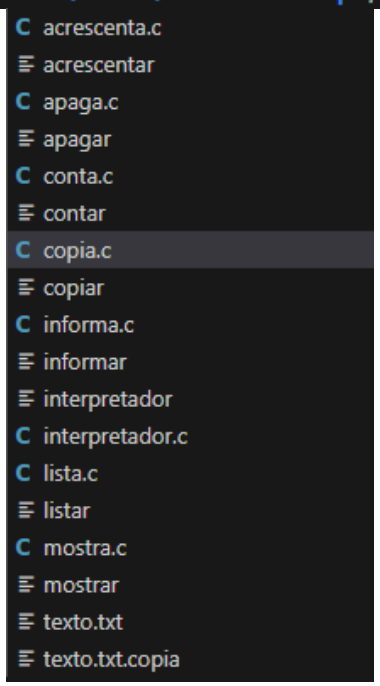
ANEXO III – IMAGENS DA EXECUÇÃO DAS FUNÇÕES

Execução mostra.c

```
ruben@DESKTOP-EPLI719:~/TP-SO/ComandosFilipe$ ./mostrar texto.txt
linha 1
linha 2
linha 3
```

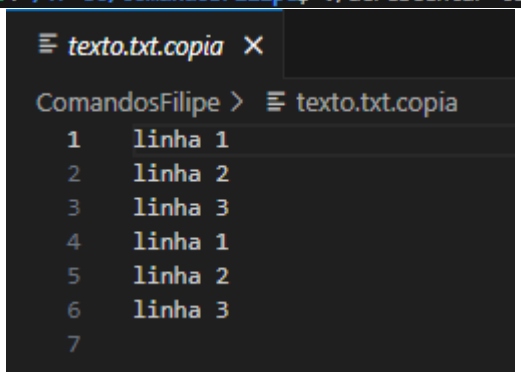
Execução copia.c

```
ruben@DESKTOP-EPLI719:~/TP-SO/ComandosFilipe$ ./copiar texto.txt
```



Execução acrescenta.c

```
ruben@DESKTOP-EPLI719:~/TP-SO/ComandosFilipe$ ./acrescentar texto.txt texto.txt.copia
```



Execução conta.c

```
ruben@DESKTOP-EPLI719:~/TP-SO/ComandosFilipe$ ./mostrar texto.txt.copia
linha 1
linha 2
linha 3
linha 1
linha 2
linha 3
ruben@DESKTOP-EPLI719:~/TP-SO/ComandosFilipe$ ./contar texto.txt.copia
Número de linhas: 6
ruben@DESKTOP-EPLI719:~/TP-SO/ComandosFilipe$
```

Execução apaga.c

```
ruben@DESKTOP-EPLI719:~/TP-SO/ComandosFilipe$ ./copiar texto.txt
ruben@DESKTOP-EPLI719:~/TP-SO/ComandosFilipe$ ./apagar texto.txt.copia
Ficheiro apagado com sucesso
ruben@DESKTOP-EPLI719:~/TP-SO/ComandosFilipe$ ls ../ComandosFilipe/
acrescenta.c  acrescentar  apaga.c  apagar  conta.c  contar  copia.c  copiar  informa.c  informar  interpretador  interpretador.c  lista.c  listar  mostra.c  mostrar  texto.t
```

Execução informa.c

```
ruben@DESKTOP-EPLI719:~/TP-SO/ComandosFilipe$ ./informar texto.txt
Tipo: Ficheiro normal
i-node: 12103423998767222
Utilizador dono: 1000
Data de criação: 2025-05-24 14:57:44
Data de última leitura: 2025-05-24 15:30:23
Data de última modificação: 2025-05-24 14:57:44
```

Execução lista.c

```
ruben@DESKTOP-EPLI719:~/TP-SO/ComandosFilipe$ ./listar ../ComandosFilipe/
.       [diretoria]
..      [diretoria]
.copia  [ficheiro]
acrescenta.c    [ficheiro]
acrescentar     [ficheiro]
apaga.c [ficheiro]
apagar  [ficheiro]
conta.c [ficheiro]
contar  [ficheiro]
copia.c [ficheiro]
copiar  [ficheiro]
informa.c       [ficheiro]
informar        [ficheiro]
interpretador   [ficheiro]
interpretador.c [ficheiro]
lista.c [ficheiro]
listar  [ficheiro]
```

ANEXO IV - IMAGEM DA EXECUÇÃO INTERPRETADOR.C

